

Study Sheets (druckfertig)

Diese Sheets sind für **Testpersonen** gedacht. Jede Rolle bekommt **ein** Sheet mit:

- **Phase 0:** Aufgaben-Stand sicherstellen (Demo/API/Seed/Gast-Workshop — je nach Code) → **nicht** von Hand nachbauen.
- **Phase 1+:** klar getrennt, ob **nur bestehende Zeilen** genutzt werden oder (nur Taskplanner) an markierten Stellen ☐ **selbst angelegt** wird.
- Spielerischer Ton; **keine** Nennung von Studienbedingungen oder konkreten Hilfs-Features — fairer Vergleich.

Die Texte **nennen keine Bedingung** (z. B. Baseline vs. anderes) und **keine internen Regelnamen** — damit der **Vergleich fair** bleibt. Sie beschreiben aber den **Ablauf**, damit adaptive Oberflächen (Banner, Anpassungen, mehrere Vorschläge) in der Praxis überhaupt sichtbar werden können.

Gast-Workshop (Codes G01 / G02)

- **Nur zwei** Gast-Konten: über „**Als Gast starten**“ vergibt der Server nacheinander G01, G02. Sind beide User-Zeilen in der DB belegt, schlägt ein weiterer Gast-Start fehl (eigener Code oder Admin-Reset der Gast-Konten).
- **Adaptiv:** Beim Start legt der Server automatisch den **Workshop** an (Kernaufgaben + **zwei Monate** verteilte Termine + **sieben** pending Beispiel-Vorschläge) — **kein** Pflicht-Schritt „Demo-Daten laden“.
- **Baseline-Gast:** derselbe Kalender-Workshop **automatisch** (Kern + Timeline, **ohne** Vorschläge). Optional: **Demo-Setup** für größeres Rollen-Set (G01/G02 only).
- **API:** POST /api/data/demo bleibt für **alle** eingeloggten Sessions nutzbar (Skripte, F-/T-/E-Codes); die UI-Karte ist nur für Gäste sichtbar.
- **Daten zurücksetzen** (Session): leert die Session; bei **G01/G02** wird der Workshop inkl. **Werk-Defaults** neu gesetzt (adaptive: alle Demo-Vorschläge wieder **pending**).
- **Admin:** „**Gast-User zurücksetzen**“ (RESET_GUEST_USERS) löscht nur G01/G02; „**Alle Demo-Testuser zurücksetzen**“ setzt F01–E05 neu **und** entfernt G01/G02.
- **Willkommen:** Demo-Story-Buttons nur für G01/G02 (optional; ersetzt die Session-Daten).

Die **Schritt-für-Schritt-Phasen** in familienplanner.md / taskplanner.md / evalrunner.md gelten für **zugeteilte** Codes (F/, T/, E/). Gast-Teilnehmer:innen: **Koordination** (dieser Abschnitt + MODERATION-UND-RUNNER.md).

Bilder & Architektur (PDF-Export)

Gemeinsame Ablage: `../assets/diagrams/` (Screenshots, exportierte Diagramme). Im Markdown reicht oft ein **kurzer Dateiname**, weil der PDF-Build über `../pdf-export/pandoc-uni-defaults.yaml` dort sucht — z.B. `![Architektur-Überblick](mein-diagramm.png)`.

Alternativ relativ zum Sheet: `![...](../assets/diagrams/mein-diagramm.png)`.

PDFs: Im Projektroot fluxplan genügt `npm run pdf` (Windows: Pandoc + MiKTeX; Details docs/pdf-export/). Nach `npm run build` werden PDFs auf Windows automatisch aktualisiert (`SKIP_DOCS_PDF=1` zum Überspringen).

Aufgaben-Sheets (eine Rolle auswählen)

- `familienplaner.md` — Familien-Captain, Konflikte & Kalender-Flow
- `taskplaner.md` — Deep-Work, Filter, Quick-Add, Erstellen
- `evalrunner.md` — System-Entdecker, Konflikte, Export/Reset

Hinter den Kulissen (nicht an Testpersonen ausgeben)

- **UI-Katalog:** alle Funktionen, **was in der UI passiert**, **Trigger**, **sichtbare Veränderungen** (Baseline vs. Adaptive): [../UI-FEATURES-KATALOG.md](#)
- **Moderation & technischer Ablauf:** Checkliste, Druckempfehlung, Verweis auf Trigger: [MODERATION-UND-RUNNER.md](#)

Technische Regelnamen und Schwellen stehen im **UI-Katalog** und in **MODERATION-UND-RUNNER.md**.

Dokumentation pflegen: Bei UI-Änderungen zuerst **UI-FEATURES-KATALOG.md** und **MODERATION-UND-RUNNER.md**. Die **Aufgaben-Sheets** anpassen, wenn sich **Navigations- oder Trigger-Abläufe** ändern (z. B. Baseline ohne Anpassungen, mehrere pending Vorschläge, Undo → wieder offen).